

High-order Proximity-aware Graph Representation Learning

Prof. O-Joun Lee

Dept. of Artificial Intelligence,
The Catholic University of Korea
ojlee@catholic.ac.kr

Contents



- High-order Proximity Overview
- Methods based on random-walks
 - DeepWalk, B. Perozzi, et al. KDD 2014.
 - LINE, J. Tang, et al. WWW 2015.
 - Node2vec, A. Grover, et al. KDD 2016.
 - APP, Zhou, Chang, et al. AAI 2017
- Methods based on matrix factorization
 - GraRep, S. Cao, et al. CIKM, 2015.
 - HOPE, M. Ou, et al. KDD 2016.
 - AROPE, Zhang, Ziwei, et al. KDD 2018

- Goal: Encode nodes $\rightarrow$ similarity in embedding space (dot product) $\approx$ similarity in the original graph

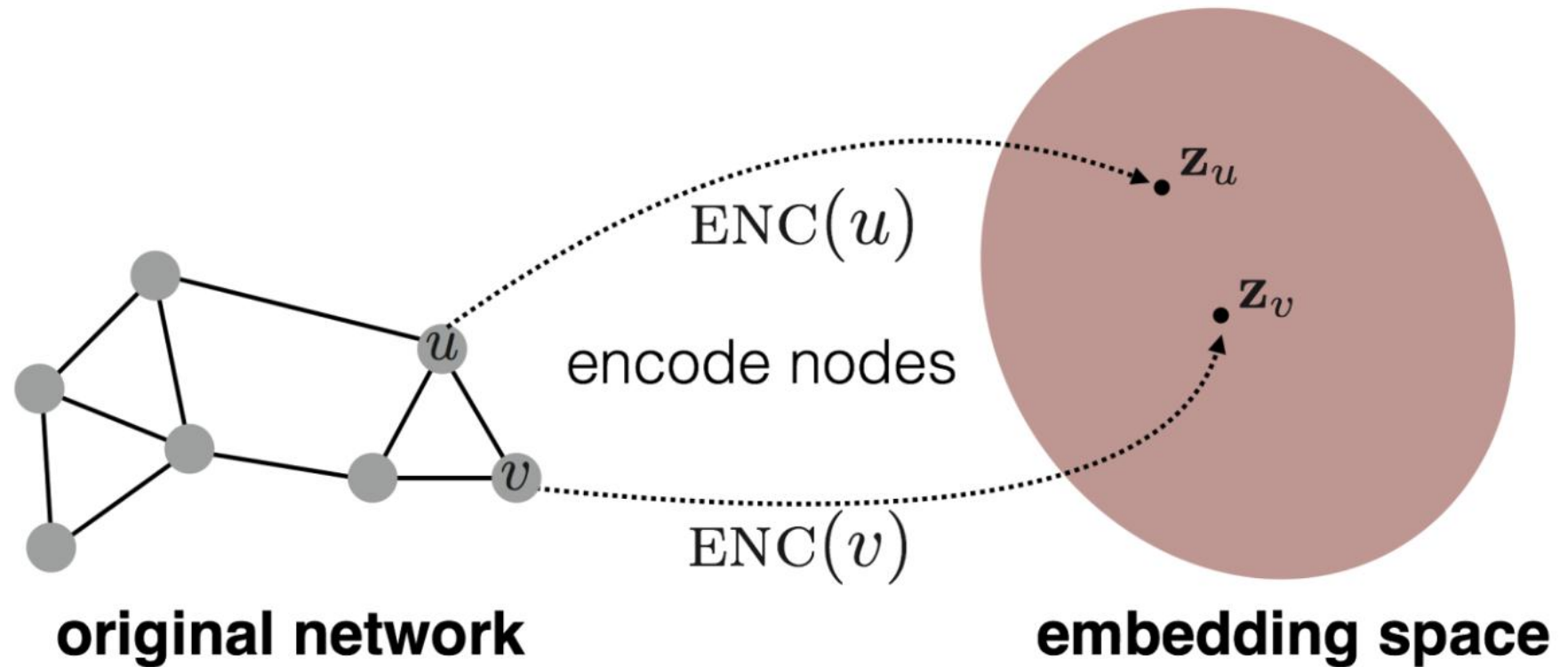
$$\text{similarity}(u, v) \approx \mathbf{z}_v^T \mathbf{z}_u$$

in the original network Similarity of the embedding

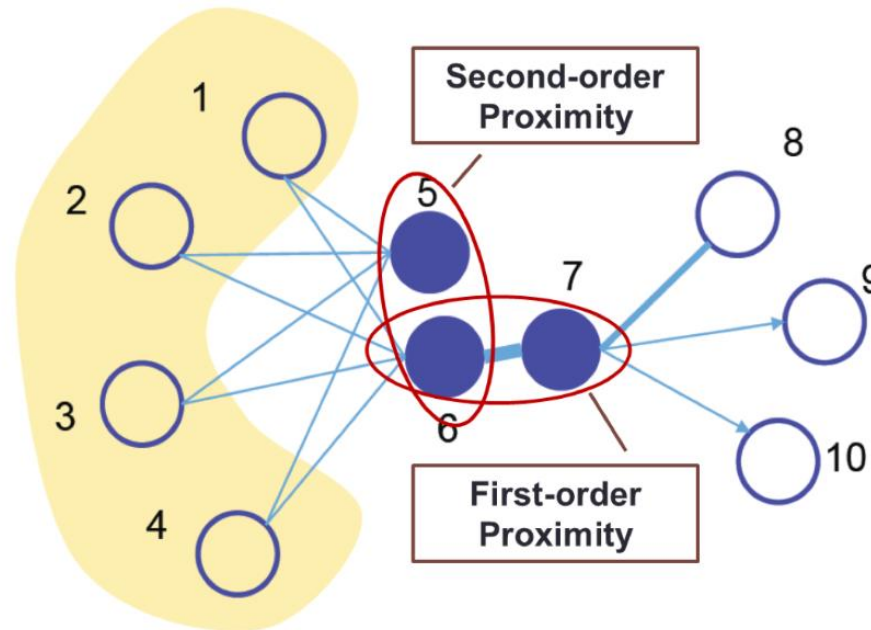
We need to define:

$\text{ENC}(u)$

$\text{Similarity}(u, v)$

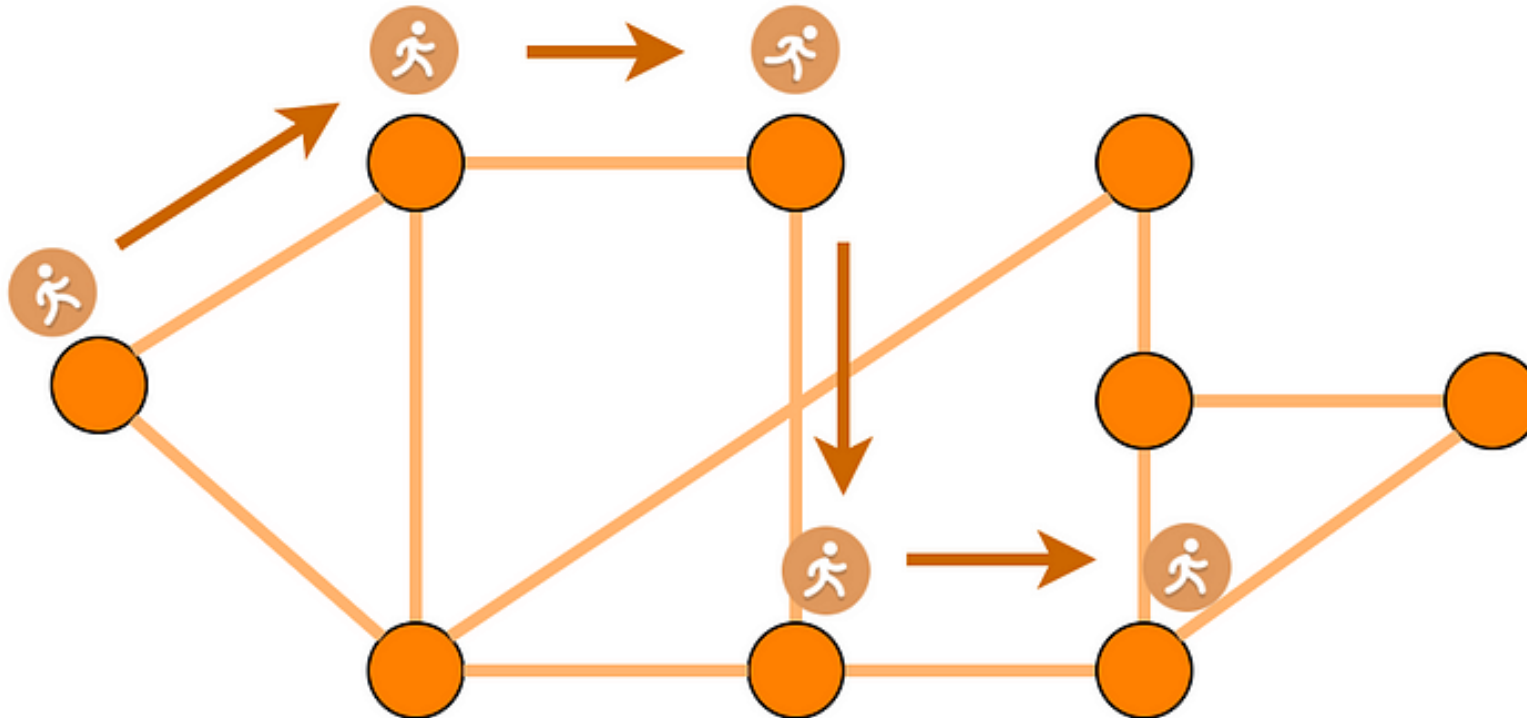


- **High-order proximity:** key in capturing the underlying structure of networks

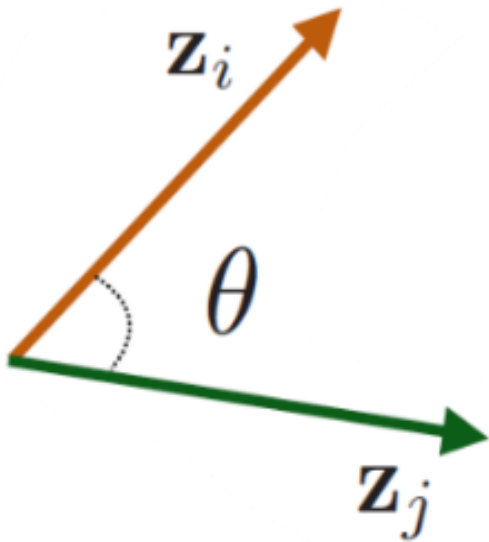


- Advantages:
 - Solve the sparsity problem of network connections
 - Measure indirect relationship between nodes

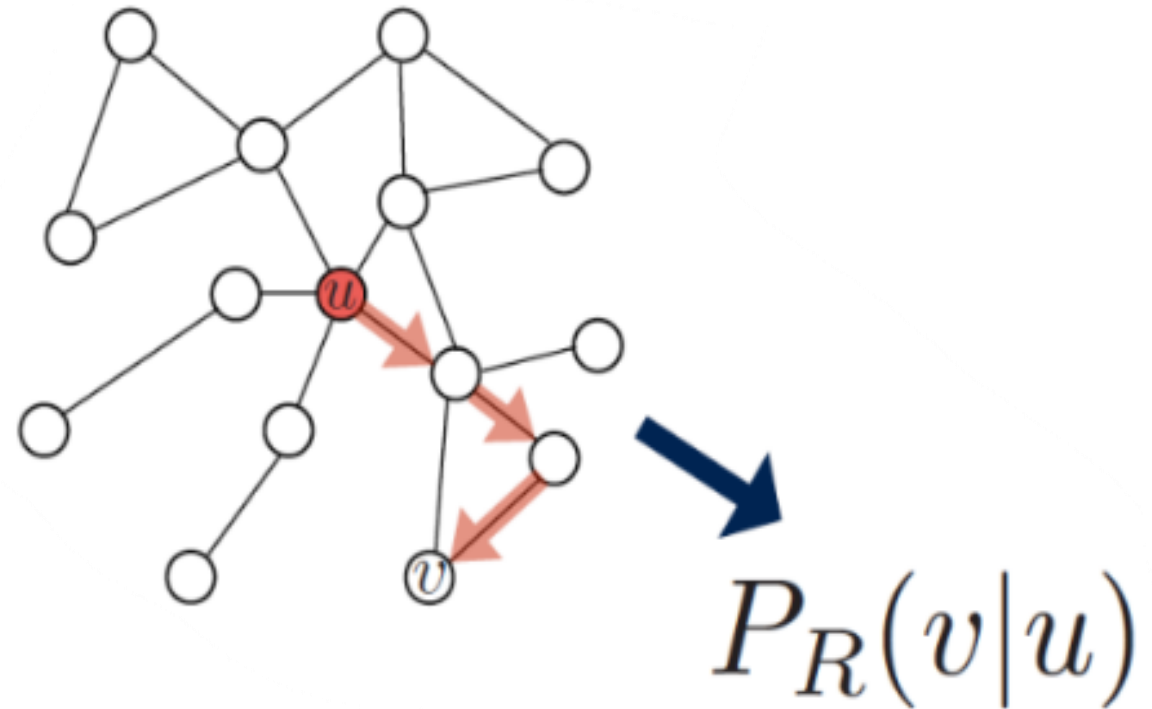
- Given a graph and a starting point, we select a neighbour of it at random, and move to this neighbour
- Then, we select a neighbor of this point at random, and move to it,...
- The random sequence of nodes visited this way is **a random walk on the graph**



- Estimate probability of visiting node v on a random walk starting from node u using some random walk strategy R
- Optimize embeddings to encode these random walk statistics



$$\mathbf{z}_u^T \mathbf{z}_v \approx$$

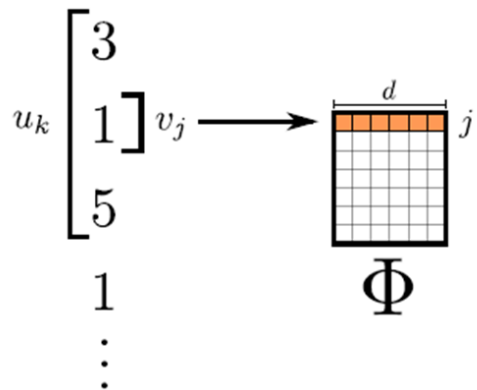


- Employ random walks on the graph to discover the structure

$v_4 \rightarrow v_3 \rightarrow v_1 \rightarrow v_5 \rightarrow v_1 \rightarrow v_{46}$

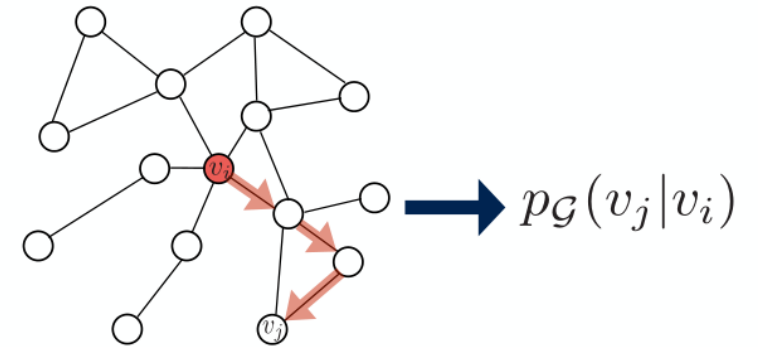
**Random walks in Network
=
Sentences in NLP**

The quick brown fox jumps over the lazy dog.

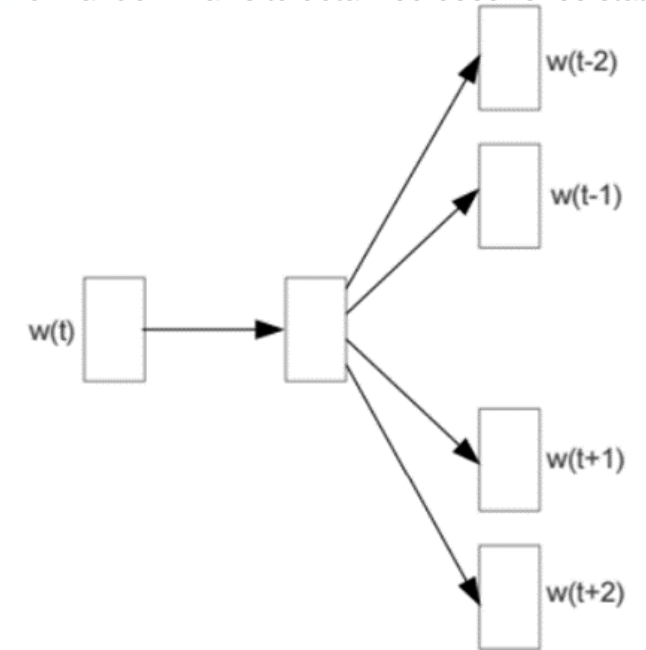


**Node sequence as the input of
word2vec models**

$$\Phi: v \in V \mapsto \mathbb{R}^{|V| \times d}$$



1. Run random walks to obtain co-occurrence statistics.



- **Goal:** Optimize

$$\mathcal{L} = \sum_{u \in V} \sum_{v \in N_R(u)} -\log(P(v|\mathbf{z}_u))$$

- (Stochastic) Gradient Descent: a simple way to minimize L
- **SGD Algorithm:** evaluate it for each individual training example
 - Initialize z_u at some randomized value for all nodes u
 - Iterative until L converges:
 - Sample a node u, for all v calculate the derivative $\frac{\partial \mathcal{L}(u)}{\partial z_v}$
 - For all v, update:

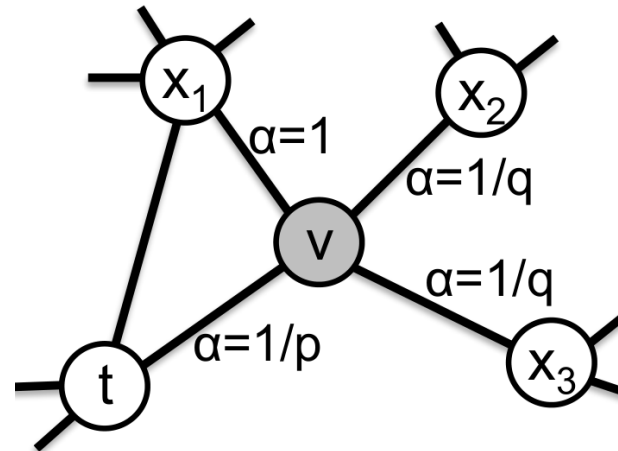
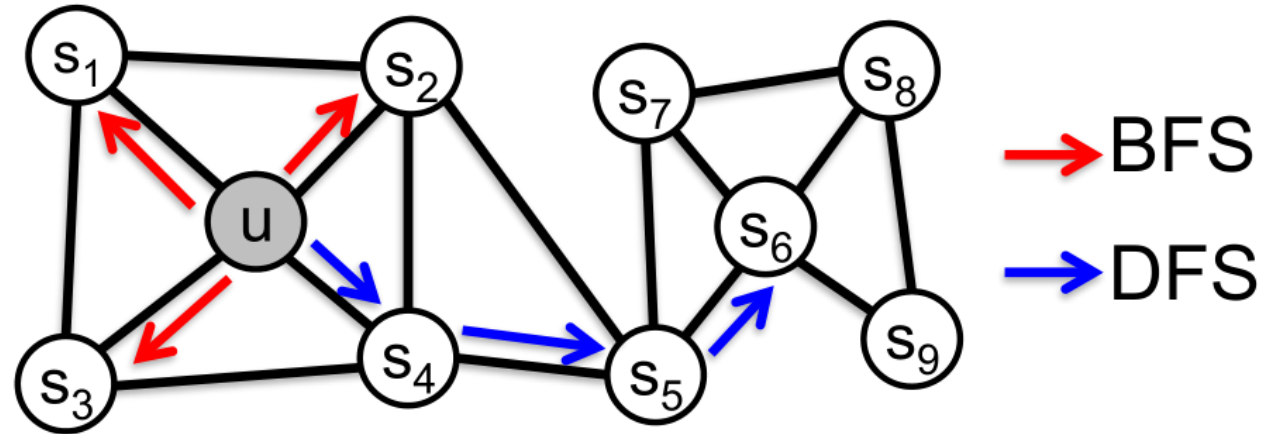
$$z_v \leftarrow z_v - \eta \frac{\partial \mathcal{L}(u)}{\partial z_v}$$

Learning rate

Biased random walks

p : controls the walk revisiting a node

q : controls the walk revisiting a node's one-hop neighborhood



$$\alpha_{pq}(t, x) = \begin{cases} \frac{1}{p} & \text{if } d_{tx} = 0 \\ 1 & \text{if } d_{tx} = 1 \\ \frac{1}{q} & \text{if } d_{tx} = 2 \end{cases}$$

p increase $\Rightarrow$ DFS

q increase $\Rightarrow$ BFS

- Problem with DeepWalk Random Walk Optimization: Expensive in summing over nodes ($O(|V|^2)$)

$$\mathcal{L} = \sum_{u \in V} \sum_{v \in N_R(u)} - \log \left(\frac{\exp(\mathbf{z}_u^T \mathbf{z}_v)}{\sum_{n \in V} \exp(\mathbf{z}_u^T \mathbf{z}_n)} \right)$$

- **Solution:** Negative Sampling (Softmax $\rightarrow$ Sigmoid)

$$\log \left(\frac{\exp(\mathbf{z}_u^T \mathbf{z}_v)}{\sum_{n \in V} \exp(\mathbf{z}_u^T \mathbf{z}_n)} \right) \approx \log \left(\sigma(\mathbf{z}_u^T \mathbf{z}_v) \right) - \sum_{i=1}^k \log \left(\sigma(\mathbf{z}_u^T \mathbf{z}_{n_i}) \right), n_i \sim P_V$$

- Sample k negative nodes n_i each with probability proportional to its degree

➤ Motivation:

- Preserving network proximities: LINE seeks to preserve both the **first-order proximity** (**direct connections between nodes**) and **second-order proximity** (**shared neighborhood structures**) in the learned embeddings
- Scalable objective functions:
 - Consider first-order and second-order proximities separately
 - Different network types: directed, undirected, weighted or unweighted
- An edge-sampling algorithm is proposed to help stochastic gradient descent on weighted edges

- First-order proximity: local pairwise proximity between two connected nodes.
- For each node pair (v_i, v_j)
 - If $(v_i, v_j) \in E$, the first-order proximity between v_i and v_j is w_{ij}
 - Otherwise, the first-order proximity between v_i and v_j is 0
- Given an undirected edge (v_i, v_j) , the joint probability of v_i and v_j :

$$p_1(v_i, v_j) = \frac{1}{1 + \exp(-\vec{u}_i^T \cdot \vec{u}_j)}$$

$\vec{u}_i$: Embedding of node v_i

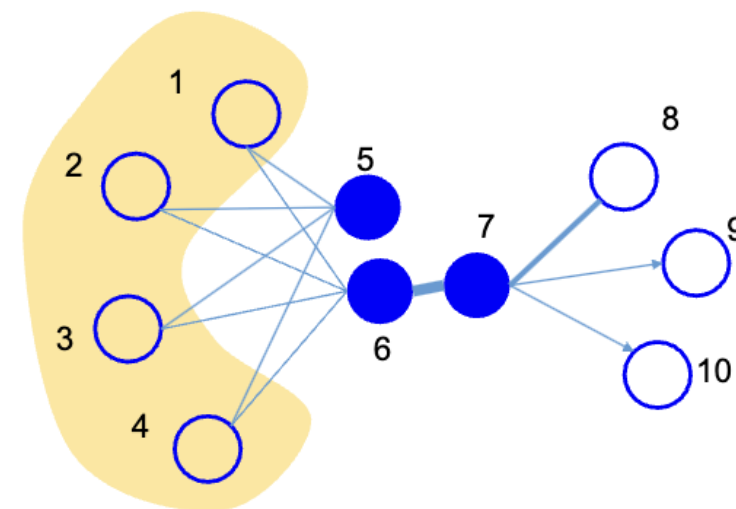
$$\hat{p}_1(v_i, v_j) = \frac{w_{ij}}{\sum_{(i', j')} w_{i' j'}}$$

- Objective:

$$O_1 = d(\hat{p}_1(\cdot, \cdot), p_1(\cdot, \cdot))$$

KL-divergence

$$\propto - \sum_{(i, j) \in E} w_{ij} \log p_1(v_i, v_j)$$

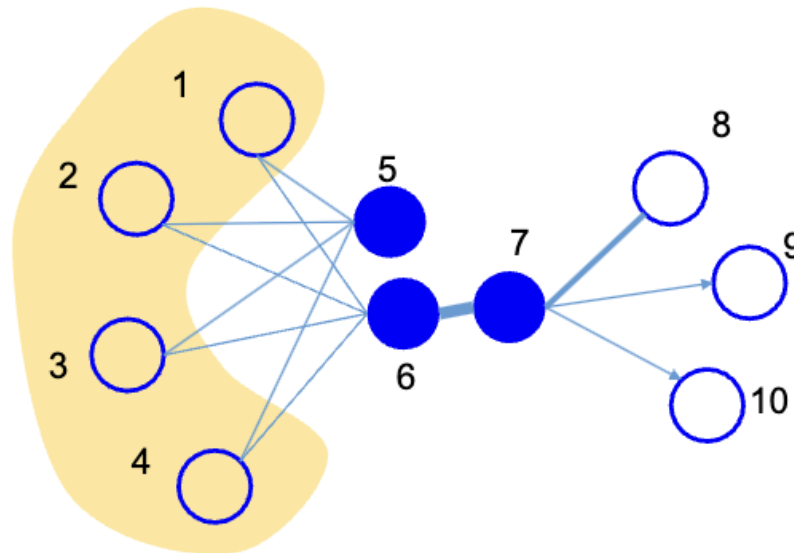


First-order: node 6 and 7

- Second-order proximity captures the 2-step relations between each pair of nodes
- For each node pair (v_i, v_j)
 - determining by the number of common neighbours shared by the two nodes

$$\hat{p}_u = (w_{u1}, w_{u2}, \dots, w_{u|V|})$$

$$\hat{p}_v = (w_{v1}, w_{v2}, \dots, w_{v|V|})$$



Second-order: node 5 and 6

$$\hat{p}_5 = (1, 1, 1, 1, 0, 0, 0, 0, 0, 0)$$

$$\hat{p}_6 = (1, 1, 1, 1, 0, 0, 5, 0, 0, 0)$$

- Given an undirected edge (v_i, v_j) , the joint probability of v_i and v_j :

$$p_2(v_j|v_i) = \frac{\exp(\vec{u}_j'^T \cdot \vec{u}_i)}{\sum_{k=1}^{|V|} \exp(\vec{u}_k'^T \cdot \vec{u}_i)},$$

$\vec{u}_i$: Embedding of node v_i when i is a source node
 $\vec{u}_i'$: Embedding of node v_i when i is a target node

$$\hat{p}_2(v_j|v_i) = \frac{w_{ij}}{\sum_{k \in V} w_{ik}}$$

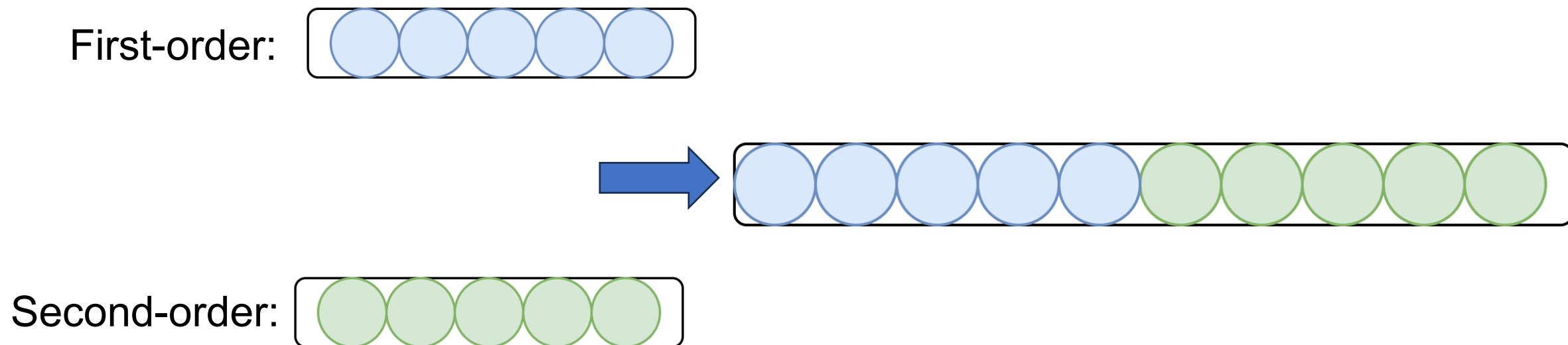
- Objective:

$$O_2 = \sum_{i \in V} \lambda_i d(\hat{p}_2(\cdot|v_i), p_2(\cdot|v_i)),$$

λ_i : Prestige of node in the network $\lambda_i = \sum_j w_{ij}$

$$\propto - \sum_{(i,j) \in E} w_{ij} \log p_2(v_j|v_i).$$

- Concatenate the embeddings individually learned by the two proximity:



- Stochastic Gradient Descent + Negative Sampling
 - Randomly sample an edge and multiple negative edges
- The gradient w.r.t the embedding with edge (i,j)

$$\frac{\partial O_2}{\partial \vec{u}_i} = w_{ij} \cdot \frac{\partial \log p_2(v_j | v_i)}{\partial \vec{u}_i}$$

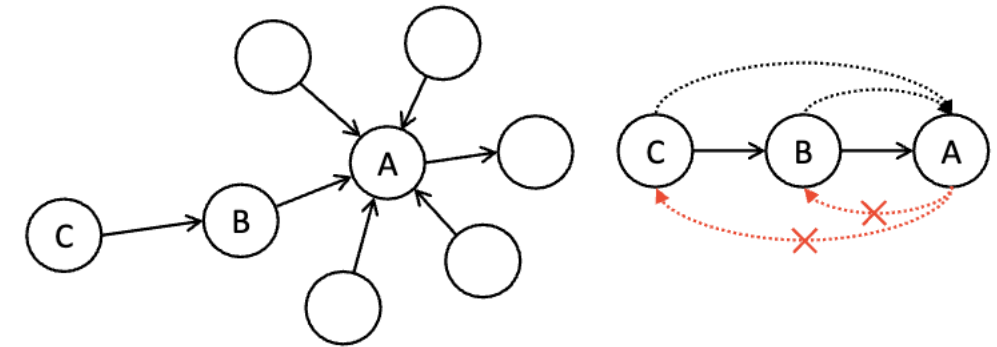
Multiplied by the weight of the edge w_{ij}

- Problematic when the weights of the edges diverge
 - The scale of the gradients with different edges diverges
- **Solution:** Edge sampling
 - Sample the edges according to their weights and treat the edges as binary
- Complexity: $\Theta(dK|E|)$
 - Linear to the dimension d , number of negative samples K , and number of edges E

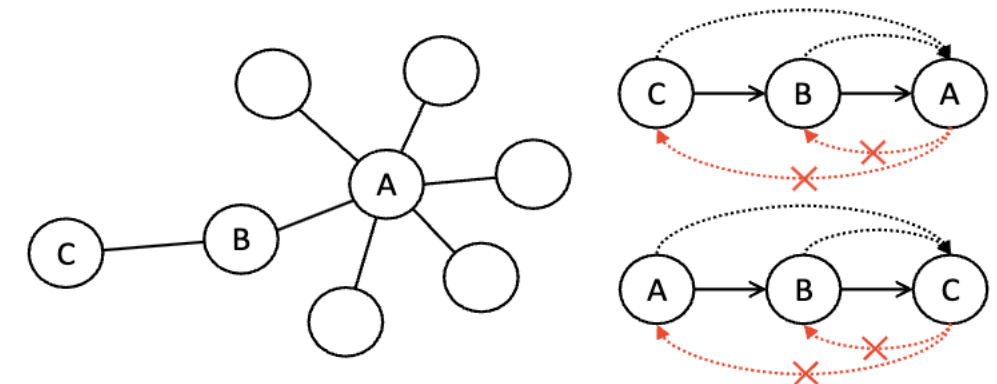
- **Initialization:** Initialize the embedding vectors for all nodes randomly
- **Edge Sampling:** Sample edges from the network based on their weights
- **Gradient Descent:** For each sampled edge, update the embedding vectors using SGD
- **Negative Sampling:** For each positive edge, sample negative edges and update the embedding vectors to maximize the difference between positive and negative samples
- **Iteration:** Repeat the edge sampling and gradient descent steps until convergence

➡ Same as DeepWalk, Node2Vec, and Node2Vec+ but consider the preservation of proximity and directed edge sampling

- Motivation: Conventional methods, i.e., DeepWalk, LINE, Node2Vec can only preserve symmetric proximities between nodes
 - Insufficient for many applications where asymmetric relationships exist, even in **undirected** graphs
- Solution: treats each path as a **directed sequence** and only observes **positive node pairs** along the forward direction of the path
 - Conventional methods have utilized random walk to sample path as an undirected sequence



(a) Directed



(b) Undirected

- **Random Walk with Restart Sampling Algorithm:** Simulate the asymmetric transition probabilities between nodes
 - Perform a random walk starting from node v , with a restart probability α at each step
 - The walk terminates when a restart occurs, and the last visited node u is returned as the endpoint
 - This sampling procedure approximates the **Rooted PageRank** score between v and u
- **Stochastic Gradient Descent Algorithm:** For each sampled path $p = v \rightarrow u$
 - Treat this as an observed positive pair (v, u) in the forward direction only
 - Optimize the following Skip-gram style objective with negative sampling

- **Path Sharing Optimization:** Propose extracting multiple node pairs from a single sampled path, by treating all suffix sub-paths or prefix sub-paths as valid sampled paths as well
- Combine Skip-gram and SGD from DeepWalk with Negative sampling from Node2vec
- Different is proposing random walk with restart sampling algorithm and treating as directed path sequence

```
# Import library
import networkx as nx
import numpy as np
from vose_sampler import VoseAlias
import matplotlib.pyplot as plt
import collections
from tqdm import tqdm
from tqdm import trange

# Create an Erdős-Rényi graph with 100 nodes and probability 0.1
G_erdos = nx.erdos_renyi_graph(100, 0.1)

# Get the edge list of the graph
edge_list = G_erdos.edges

# Create a dictionary to store the edge list with weights and total weightsum
edgedistdict = collections.defaultdict(int)
weightsum = 0

# For each edge in the edge list, assign a random weight between 1 and 100.
for edge in edge_list:
    weight = np.random.uniform(1, 100)
    edgedistdict[(edge[0], edge[1])] = weight
    weightsum += weight

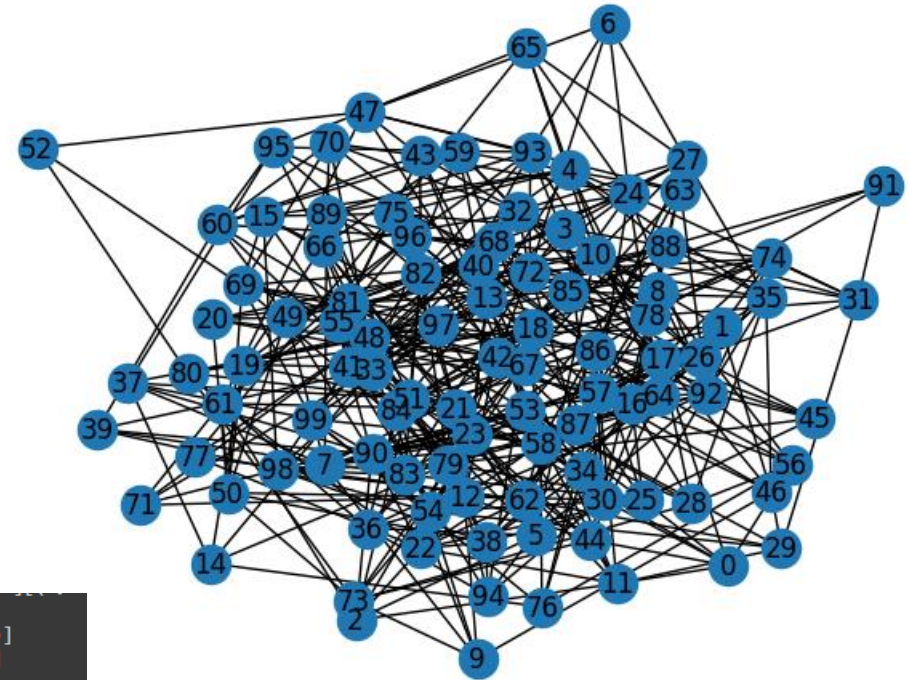
# Print the normalized edge list with weights
print(edgedistdict)

defaultdict(<class 'int'>, {(0, 12): 0.0022917282387984626, (0, 25): 0.001658375...

# Edge sampling using alias table
edgesaliassampler = VoseAlias(edgedistdict)
batchrange = int(len(edgedistdict) / 5)

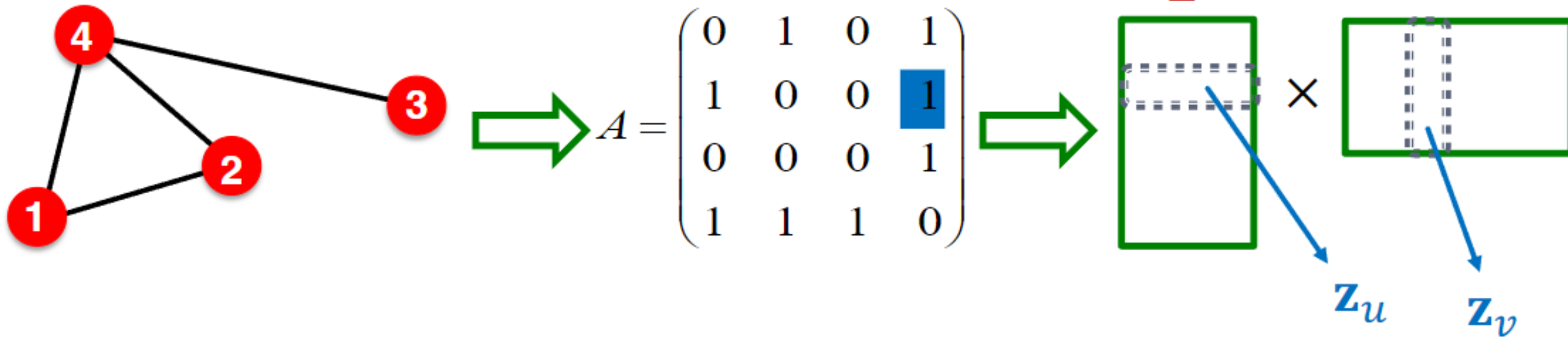
for b in trange(batchrange):
    print(edgesaliassampler.sample_n(size=5))
```

```
[(3, 8), (37, 93), (21, 63), (67, 94), (42, 79)]
[(22, 31), (50, 57), (12, 71), (44, 66), (27, 70)]
[(45, 71), (50, 55), (10, 74), (2, 22), (15, 71)]
[(4, 26), (21, 53), (22, 23), (5, 41), (11, 24)]
[(29, 64), (17, 51), (31, 69), (12, 22), (35, 63)]
[(17, 18), (59, 75), (15, 95), (43, 76), (0, 66)]
[(3, 22), (45, 71), (75, 93), (7, 75), (21, 98)]
[(11, 80), (1, 62), (31, 61), (0, 68), (12, 19)]
[(90, 95), (0, 60), (17, 18), (12, 20), (52, 99)]
[(38, 46), (27, 88), (22, 73), (24, 54), (0, 93)]
[(52, 74), (52, 99), (13, 70), (11, 47), (20, 45)]
[(15, 71), (12, 22), (0, 66), (5, 11), (64, 89)]
[(10, 74), (7, 97), (69, 74), (58, 90), (48, 89)]
[(38, 44), (4, 39), (50, 57), (7, 75), (14, 97)]
[(7, 15), (13, 18), (25, 50), (25, 54), (38, 44)]
[(0, 81), (90, 95), (15, 64), (32, 55), (21, 98)]
[(11, 18), (2, 47), (22, 83), (13, 26), (9, 78)]
[(22, 73), (62, 63), (38, 44), (35, 81), (6, 27)]
[(47, 75), (21, 53), (7, 15), (51, 92), (13, 84)]
[(11, 47), (2, 96), (43, 45), (29, 30), (18, 78)]
[(94, 99), (33, 61), (45, 85), (41, 46), (15, 95)]
[(69, 82), (79, 82), (9, 56), (21, 63), (18, 40)]
[(23, 94), (69, 80), (69, 82), (33, 66), (69, 74)]
[(59, 62), (20, 27), (3, 84), (79, 81), (40, 63)]
```



- One of the simplest and most intuitive approaches to defining similarity:
 - Adjacency between two nodes v and u
- Similarity: Two nodes are adjacent to one another within the structure of the graph

$$\mathbf{z}_v^T \mathbf{z}_u = A_{u,v} \text{ or } \mathbf{Z}^T \mathbf{Z} = A$$



- Encoding: Find the embedding matrix $\mathbf{Z}$ that minimizes the loss function $\mathcal{L}$

The diagram illustrates the loss function $\mathcal{L}$ used in adjacency-based approaches. It is defined as the sum over all node pairs $(u, v) \in V \times V$ of the squared difference between the embedding similarity $\|\mathbf{z}_u^\top \mathbf{z}_v\|$ and the (weighted) adjacency matrix entry $\mathbf{A}_{u,v}$. The components are labeled as follows:

- $\mathcal{L}$: loss (what we want to minimize)
- $\sum_{(u,v) \in V \times V}$: sum over all node pairs
- $\|\mathbf{z}_u^\top \mathbf{z}_v\|$: embedding similarity
- $\mathbf{A}_{u,v}$: (weighted) adjacency matrix for the graph

$$\mathcal{L} = \sum_{(u,v) \in V \times V} \|\mathbf{z}_u^\top \mathbf{z}_v - \mathbf{A}_{u,v}\|^2$$

- Solution:
 - Option 1: Use stochastic gradient descent (SGD) as a general optimization method
 - Option 2: Solve matrix decomposition solvers (e.g., SVD or QR decomposition routines)
 - **Matrix Factorization-based**

- Let A be an $m \times n$ adjacency matrix. The factorization of A takes the form

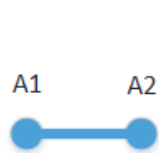
$$A = USV^T$$

where U is a $m \times m$ orthogonal matrix, V^T is a $n \times n$ orthogonal matrix and S is a $m \times n$ diagonal matrix

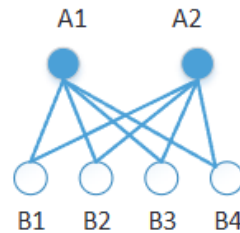
- Factorize to low-rank approximations based on minimizing the sum-squared distance using **Singular Value Decomposition**
- To decompose:
 - Evaluate the n eigenvectors v_i and eigenvalues λ_i of $A^T A$
 - Make a matrix V from the normalized vectors v_i
 - Make a diagonal matrix S from the square roots of the eigenvalues

$$\sigma_i = \sqrt{\lambda_i} \quad \text{and} \quad \sigma_1 \geq \sigma_2 \geq \sigma_3 \dots$$

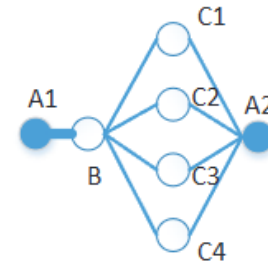
- Find U : $A = USV^T \Rightarrow US = AV \Rightarrow U = AVS^{-1}$



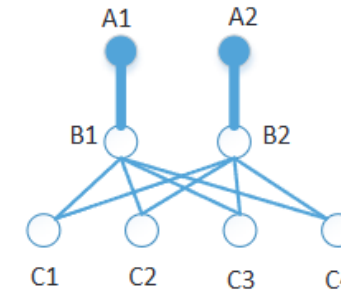
(a) 1-hop relation



(b) 2-hop relation

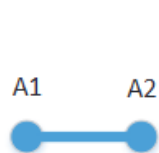


(c) 3-hop relation

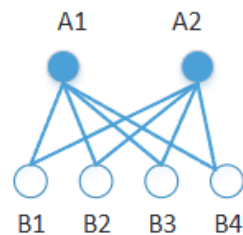


(d) 4-hop relation

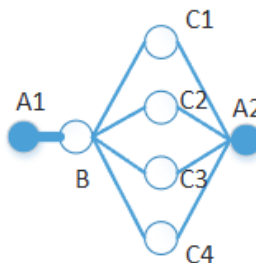
- Use the skip-gram model to capture the k -step ($k = 1, 2, 3, \dots$) relationships between each vertex and its k -step neighbors in the graph with different values of k ($k = 1, 2, 3$)
- Please review the **skip-gram model** for text
- Here, the objectives are to learn the representation of the graph nodes based on
 - Long-distance relationship between two different nodes
 - The consideration of distinct connections in terms of different transitional steps



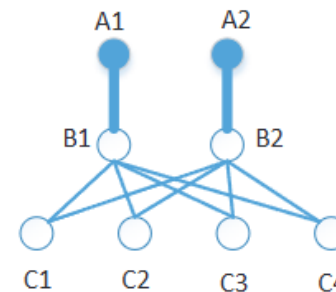
(a) 1-hop relation



(b) 2-hop relation



(c) 3-hop relation



(d) 4-hop relation

$$p_k(c|w) = A_{w,c}^k \quad p_k(c) = \sum_{w'} q(w') p_k(c|w') = \frac{1}{N} \sum_{w'} A_{w',c}^k$$

$$L_k(w, c) = A_{w,c}^k \cdot \log \sigma(\vec{w} \cdot \vec{c}) + \frac{\lambda}{N} \sum_{w'} A_{w',c}^k \cdot \log \sigma(-\vec{w} \cdot \vec{c})$$

$$\vec{w} \cdot \vec{c} = \log \left(\frac{A_{w,c}^k}{\sum_{w'} A_{w',c}^k} \right) - \log(\beta)$$

$$Y_{i,j}^k = W_i^k \cdot C_j^k = \log \left(\frac{A_{i,j}^k}{\sum_t A_{t,j}^k} \right) - \log(\beta)$$

Practical computation of multi-hop neighbour matrix

- Use log-transformed, probabilistic adjacency matrix
- Train embeddings to predict k-hop neighbors
- Train multiple hop values, and concatenate output embeddings

$$\tilde{A}_{i,j}^k = \max \left(\log \left(\frac{(A_{i,j}^k / d_i)}{\sum_{l \in V} (A_{l,j}^k / d_l)} \right) - \alpha, 0 \right)$$

Adjacency matrix

Node Degree

Table 1: Overall Algorithm

GraRep Algorithm
Input Adjacency matrix S on graph Maximum transition step K Log shifted factor β Dimension of representation vector d
1. Get k-step transition probability matrix A^k Compute $A = D^{-1}S$ Calculate $A^1, A^2, \dots, A^K$, respectively 2. Get each k-step representations For $k = 1$ to K 2.1 Get positive log probability matrix calculate $\Gamma_1^k, \Gamma_2^k, \dots, \Gamma_N^k$ ($\Gamma_j^k = \sum_p A_{p,j}^k$) respectively calculate $\{X_{i,j}^k\}$ $X_{i,j}^k = \log \left(\frac{A_{i,j}^k}{\Gamma_j^k} \right) - \log(\beta)$ assign negative entries of X^k to 0 2.2 Construct the representation vector W^k $[U^k, \Sigma^k, (V^k)^T] = SVD(X^k)$ $W^k = U_d^k (\Sigma_d^k)^{\frac{1}{2}}$ 3. Concatenate all the k-step representations $W = [W^1, W^2, \dots, W^K]$
Output Matrix of the graph representation W

probability transition matrix

represent the probability of moving from node i to node j in exactly k steps

representation matrix W^k , where each row is a d -dimensional vector capturing k -step relational information.

- Idea: Critical property in directed graph - **Asymmetric Transitivity**
- Transitivity is **Asymmetric** in directed graph
- Challenge: Incorporate asymmetric transitivity in graph embedding
- Problem: Metric space is **symmetric**

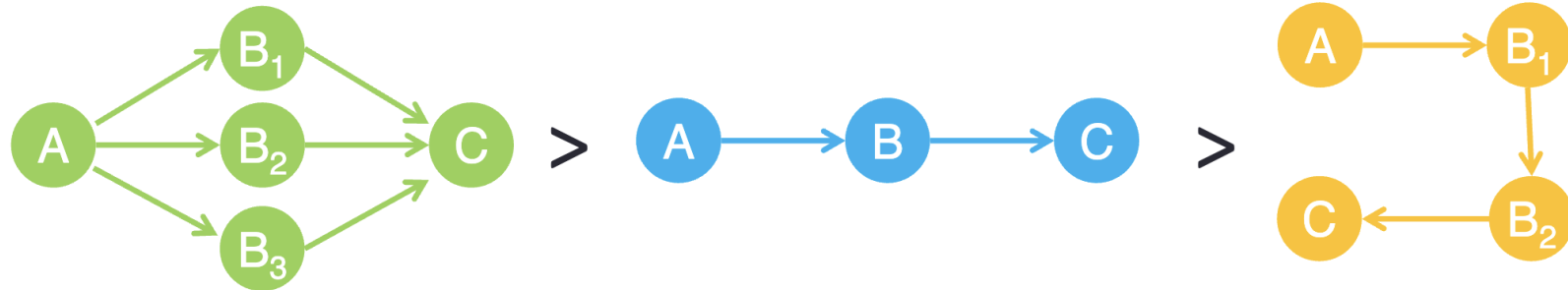


➤ High-order Proximity with asymmetric transitivity:

➤ Asymmetry: not symmetric in directed graph

➤ Transitivity:

- More directed paths, larger similarity
- Shorter paths, larger similarity



➤ Compare $A \rightarrow C$ similarity: Katz Index

$$\mathbf{s}^{Katz} = \sum_{l=1}^{+\infty} (\beta \cdot A)^l$$

➤ Preserve high-order proximity embedding:

➤ Katz Index modified:

$$\mathbf{S}^{Katz} = \sum_{l=1}^{+\infty} (\beta \cdot A)^l = (I - \beta \cdot A)^{-1} \cdot (\beta \cdot A)$$

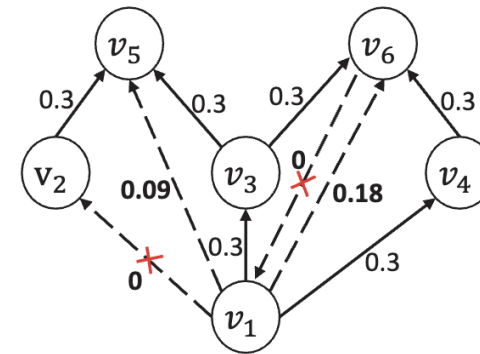
➤ Objective:

$$\min_{U_s, U_t} \|\mathbf{S} - U_s \cdot U_t^T\|_F^2$$

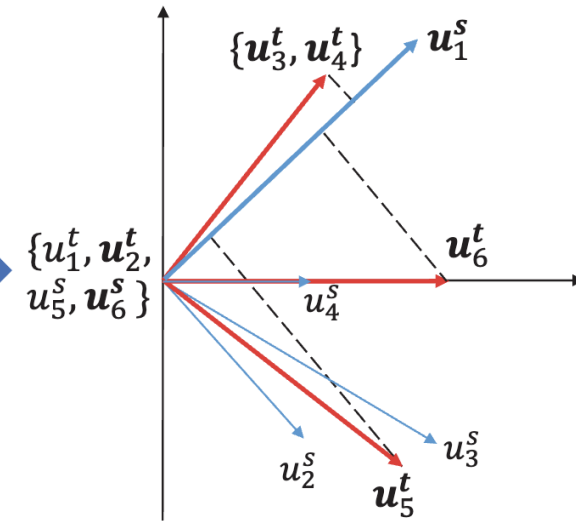
$$\mathbf{S} = M_g^{-1} \cdot M_l$$

Embedding similarity

where M_g, M_l are polynomial of adjacency matrix or proximity measurements: Katz, Adamic Adar, PageRank, Common Neighbors



HOPE



➤ Solving by **Generalized Singular Value Decomposition**: decompose $\mathbf{S}$ without calculating it

➤ **Linear complexity** w.r.t. the volume of data (i.e. edge number)

➤ Objective:

$$\min_{U_s, U_t} \|S - U_s \cdot U_t^T\|_F^2$$

$$S = M_g^{-1} \cdot M_l$$

➤ If we have the singular value decomposition of the general formulation:

$$M_g^{-1} \cdot M_l = V^s \Sigma V^{t\top} \text{ ————— } V^t \text{ and } V^s: \text{orthogonal matrices.}$$

$$\Sigma = \text{diag}(\sigma_1, \sigma_2, \dots, \sigma_N) \text{ ————— } \text{Non-negative diagonal matrix.}$$

➤ There exists a nonsingular matrix X and two diagonal matrices, i.e. Σ^l and Σ^g , satisfying

$$V^{t\top} M_l^\top X = \Sigma^l \quad \text{where } \Sigma^l = \text{diag}(\sigma_1^l, \sigma_2^l, \dots, \sigma_N^l)$$

$$V^{s\top} M_g^\top X = \Sigma^g \quad \Sigma^g = \text{diag}(\sigma_1^g, \sigma_2^g, \dots, \sigma_N^g)$$

$$\sigma_1^l \geq \sigma_2^l \geq \dots \geq \sigma_K^l \geq 0$$

$$0 \leq \sigma_1^g \leq \sigma_2^g \leq \dots \leq \sigma_K^g$$

$$\forall i \quad \sigma_i^{l^2} + \sigma_i^{g^2} = 1$$

$$\sigma_i = \frac{\sigma_i^l}{\sigma_i^g}$$

➤ **Linear complexity** w.r.t. the volume of data (i.e. edge number)

$$O(K^2 L \cdot m)$$

Embedding Dimension (constant) Iteration (constant) Edge number

Algorithm 1 High-order Proximity preserved Embedding

Require: adjacency matrix $\mathbf{A}$, embedding dimension K , parameters of high-order proximity measurement θ .

Ensure: embedding source vectors $\mathbf{U}^s$ and target vectors $\mathbf{U}^t$.

- 1: calculate $\mathbf{M}_g$ and $\mathbf{M}_l$.
 - 2: perform JDGSVD with $\mathbf{M}_g$ and $\mathbf{M}_l$, and obtain the generalized singular values $\{\sigma_1^l, \dots, \sigma_K^l\}$ and $\{\sigma_1^g, \dots, \sigma_K^g\}$, and the corresponding singular vectors, $\{\mathbf{v}_1^s, \dots, \mathbf{v}_K^s\}$ and $\{\mathbf{v}_1^t, \dots, \mathbf{v}_K^t\}$.
 - 3: calculate singular values $\{\sigma_1, \dots, \sigma_K\}$ according to Equation (21).
 - 4: calculate embedding matrices $\mathbf{U}^s$ and $\mathbf{U}^t$ according to Equation (19) and (20).
-

JDGSVD: Iteration of Jacobi-Davidson
Generalized Singular Value Decomposition

No need Stochastic Gradient Descent optimization

Focus on solving matrix factorization

HOPE's problems:

- **Fixed Proximity Form:** HOPE requires high-order proximities to be in a very specific "*generalized SVD*" form to maintain linear complexity. It cannot preserve proximities of arbitrary orders or weights outside of that specific mathematical structure.
- **Lack of Flexibility:** HOPE cannot shift across different orders or efficiently re-calculate embeddings if the user wants to experiment with different proximity settings. → AROPE treats HOPE as a special case where the order is set to infinity and weights are exponentially decaying.

S is defined as a *polynomial* function of the adjacency matrix A . It combines different "orders" of connections to capture both direct and indirect relationships:

$$S = F(A) = w_1A + w_2A^2 + \dots + w_qA^q$$

where:

- A^1 (the adjacency matrix) represents direct neighbors (1st-order).
- A^2 represents nodes reachable in two steps (2nd-order, like common neighbors).
- A^q represents nodes reachable in q steps (q -th order).
- w_i are weights that determine the importance of each specific order.

Eigen-Decomposition Reweighting is the core theorem of AROPE that reveals how proximities of different orders are related through their eigenvalues.

Core Principle

The theorem states that if (λ, x) is an **eigen-pair** of the adjacency matrix A , then $(F(\lambda), x)$ is an eigen-pair of the proximity matrix S .

- **Eigenvectors (x):** Remain identical for any proximity order. They act as the "coordinates" for nodes.
- **Eigenvalues (λ):** Are transformed by the proximity function $F(\cdot)$. They act as the "*weights*" for each coordinate.

Significance

This finding allows the model to compute the eigen-decomposition of the sparse adjacency matrix A just once. To get the embeddings for any proximity order q , the algorithm simply updates the eigenvalues using the polynomial function:

$$F(\lambda) = w_1\lambda + w_2\lambda^2 + \dots + w_q\lambda^q$$

This makes shifting between different proximity weights extremely fast, as it only requires re-sorting and re-scaling the **pre-computed eigenvectors**.

By pre-computing the top eigen-decomposition of the adjacency matrix, the method can shift across arbitrary orders with minimal marginal cost.

Algorithm 1 AROPE: ARbitrary-Order Proximity preserved Embedding

Require: Adjacency Matrix A , Dimensionality d , Different High-Order Proximity Functions $\mathcal{F}_1(\cdot), \dots, \mathcal{F}_r(\cdot)$

Ensure: Embedding vectors U_i^*, V_i^* for $\mathcal{F}_i(\cdot)$, $1 \leq i \leq r$

- 1: Calculate the top- l eigen-decomposition $[\Lambda, X]$ of A
 - 2: **for** i in $1:r$ **do**
 - 3: Calculate the reweighted eigenvalues $\Lambda' = \mathcal{F}_i(\Lambda)$
 - 4: Sort Λ' in descending order of the absolute value and select the top- d
 - 5: Calculate the top- d SVD results using Eq. (4)
 - 6: Return U_i^*, V_i^* using Eq. (3)
 - 7: **end for**
-

Initial Eigen-Decomposition: The algorithm first calculates the top- l eigenvalues (Λ) and eigenvectors (X) of the sparse adjacency matrix A . This is the most computationally expensive step but is only performed **once**.

Algorithm 1 AROPE: ARbitrary-Order Proximity preserved Embedding

Require: Adjacency Matrix A , Dimensionality d , Different High-Order Proximity Functions $\mathcal{F}_1(\cdot), \dots, \mathcal{F}_r(\cdot)$

Ensure: Embedding vectors U_i^*, V_i^* for $\mathcal{F}_i(\cdot)$, $1 \leq i \leq r$

- 1: Calculate the top- l eigen-decomposition $[\Lambda, X]$ of A
 - 2: **for** i in $1:r$ **do**
 - 3: Calculate the reweighted eigenvalues $\Lambda' = \mathcal{F}_i(\Lambda)$
 - 4: Sort Λ' in descending order of the absolute value and select the top- d
 - 5: Calculate the top- d SVD results using Eq. (4)
 - 6: Return U_i^*, V_i^* using Eq. (3)
 - 7: **end for**
-

Reweight Eigenvalues: It applies the polynomial function to the pre-computed eigenvalues: $\Lambda' = \mathcal{F}_i(\Lambda)$. This transforms the 1st-order importance into high-order importance.

Algorithm 1 AROPE: ARbitrary-Order Proximity preserved Embedding

Require: Adjacency Matrix A , Dimensionality d , Different High-Order Proximity Functions $\mathcal{F}_1(\cdot), \dots, \mathcal{F}_r(\cdot)$

Ensure: Embedding vectors U_i^*, V_i^* for $\mathcal{F}_i(\cdot)$, $1 \leq i \leq r$

- 1: Calculate the top- l eigen-decomposition $[\Lambda, X]$ of A
 - 2: **for** i in $1:r$ **do**
 - 3: Calculate the reweighted eigenvalues $\Lambda' = \mathcal{F}_i(\Lambda)$
 - 4: Sort Λ' in descending order of the absolute value and select the top- d
 - 5: Calculate the top- d SVD results using Eq. (4)
 - 6: Return U_i^*, V_i^* using Eq. (3)
 - 7: **end for**
-

Sort and Select: The new eigenvalues Λ' are sorted by absolute value, and the top- d dimensions are selected. This step ensures the most significant "reweighted" features are used.

Algorithm 1 AROPE: ARbitrary-Order Proximity preserved Embedding

Require: Adjacency Matrix A , Dimensionality d , Different High-Order Proximity Functions $\mathcal{F}_1(\cdot), \dots, \mathcal{F}_r(\cdot)$

Ensure: Embedding vectors U_i^*, V_i^* for $\mathcal{F}_i(\cdot)$, $1 \leq i \leq r$

- 1: Calculate the top- l eigen-decomposition $[\Lambda, X]$ of A
 - 2: **for** i in $1:r$ **do**
 - 3: Calculate the reweighted eigenvalues $\Lambda' = \mathcal{F}_i(\Lambda)$
 - 4: Sort Λ' in descending order of the absolute value and select the top- d
 - 5: Calculate the top- d SVD results using Eq. (4)
 - 6: Return U_i^*, V_i^* using Eq. (3)
 - 7: **end for**
-

SVD Conversion: It converts these eigen-decomposition results into SVD components (singular values and left/right singular vectors).

THEOREM 4.1. For any symmetric matrix S , $\forall 1 \leq i \leq d$, we have:

$$\begin{cases} U(:, i) = X(:, i) \\ \Sigma(i, i) = \text{abs}(\Lambda(i, i)) \end{cases}, \text{ and} \quad (4)$$

$$\begin{cases} X(:, i) = U(:, i) \\ \Lambda(i, i) = \Sigma(i, i) \text{sign}(U(:, i) \cdot V(:, i)) \end{cases} \quad (5)$$

where $\text{abs}(x) = |x|$ stands for the absolute value function and $\text{sign}(\cdot)$ is the sign function, i.e. $\text{sign}(x) = 1$ if $x > 0$, $\text{sign}(x) = 0$ if $x = 0$ and $\text{sign}(x) = -1$ if $x < 0$.

Algorithm 1 AROPE: ARbitrary-Order Proximity preserved Embedding

Require: Adjacency Matrix A , Dimensionality d , Different High-Order Proximity Functions $\mathcal{F}_1(\cdot), \dots, \mathcal{F}_r(\cdot)$

Ensure: Embedding vectors U_i^*, V_i^* for $\mathcal{F}_i(\cdot)$, $1 \leq i \leq r$

- 1: Calculate the top- l eigen-decomposition $[\Lambda, X]$ of A
 - 2: **for** i in $1:r$ **do**
 - 3: Calculate the reweighted eigenvalues $\Lambda' = \mathcal{F}_i(\Lambda)$
 - 4: Sort Λ' in descending order of the absolute value and select the top- d
 - 5: Calculate the top- d SVD results using Eq. (4)
 - 6: Return U_i^*, V_i^* using Eq. (3)
 - 7: **end for**
-

The final *content* (U^*) and *context* (V^*) embedding vectors are obtained by scaling the eigenvectors by the square root of the singular values.

$$U^* = U\sqrt{\Sigma} \quad \text{and} \quad V^* = V\sqrt{\Sigma}$$

- **U^* (Content Vectors):** These represent the nodes themselves.
- **V^* (Context Vectors):** These represent the "neighborhood" or context of the nodes.

```
import networkx as nx
import numpy as np
from scipy.sparse.linalg import svds
from scipy.sparse import csr_matrix

# Load the karate club network
G = nx.karate_club_graph()

# Define the Katz index similarity
def katz_index(G, alpha=0.5, max_iter=100, tol=1e-3):
    n = len(G.nodes())
    A = nx.adjacency_matrix(G).todense()
    I = np.eye(n)
    X = I
    converged = False
    for i in range(max_iter):
        X_prev = X
        X = alpha * A.dot(X) + (1 - alpha) * I
        if np.linalg.norm(X - X_prev) < tol:
            converged = True
            break
    return X

# Compute the Katz index similarity matrix
X = katz_index(G)

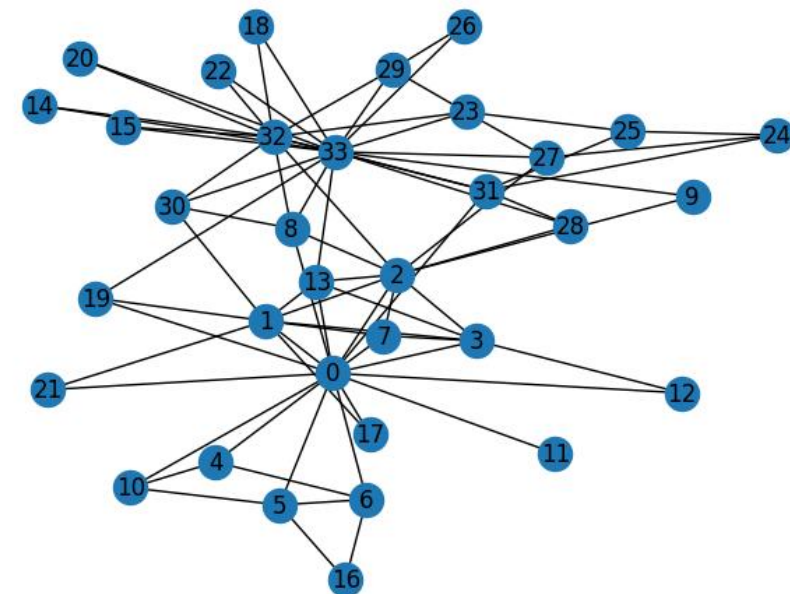
# Perform singular value decomposition on the adjacency matrix
U, s, Vh = svds(X, k=2)

# Print the singular values
print(s)

# Print the left singular vectors
print(U)

# Print the right singular vectors
print(Vh)
```

```
[2.14015007e+092 3.46416835e+103]
[[ 0.33091154 -0.31235123]
 [ 0.33411655 -0.30175117]
 [ 0.27459589 -0.36058862]
 [ 0.24818157 -0.19856719]
 [ 0.08309555 -0.05567475]
 [ 0.10827277 -0.06742808]
 [ 0.10598536 -0.06644822]
 [ 0.22455028 -0.17843234]
 [ 0.03319046 -0.24695058]
 [-0.02345542 -0.05020307]
 [ 0.07224995 -0.04583322]
 [ 0.05803347 -0.04320695]
 [ 0.06286895 -0.04186974]
 [ 0.18811321 -0.24048654]
 [-0.09784877 -0.07953765]
 [-0.13735606 -0.1131142 ]
 [ 0.03757528 -0.01851886]
 [ 0.05822106 -0.04271819]
 [-0.05895455 -0.04889692]
 [ 0.05799926 -0.07342003]
 [-0.07809517 -0.06274937]
 [ 0.07775279 -0.05663175]
 [-0.09815528 -0.08100556]
 [ 0.28970329 -0.21804371]
 [-0.06604098 -0.05104891]
 [-0.17460707 -0.12250229]
 [-0.08337386 -0.05862926]
 [-0.12623386 -0.14768307]
 [-0.030889142 -0.08614539]
 [-0.18759871 -0.13583302]
 [-0.08436143 -0.15831325]
 [-0.20089774 -0.20945644]
 [-0.332677 -0.33226141]
 [-0.33791409 -0.36409688]]
[[ 0.18797699 -0.05775017 -0.05189329 0.20018482 -0.05997669 -0.03472371
 0.07575835 -0.01988671 -0.12292961 0.04664735 -0.37814548 -0.05137177
 0.20501379 0.33504652 0.31253281 0.14576309 0.03485069 0.02443477
 -0.1227842 -0.13729933 0.03702881 -0.28451102 0.00194154 -0.01871236
 -0.31841159 0.16264699 0.07591105 0.05179984 0.02405201 0.27974509
 -0.17606471 0.00428549 -0.05424557 -0.33020121]
```





네트워크 과학연구실
NETWORK SCIENCE LAB



가톨릭대학교
THE CATHOLIC UNIVERSITY OF KOREA

